```
template<typename Iterator,typename T>
T parallel_accumulate(Iterator first,Iterator last,T init)
{
    unsigned long const length=std::distance(first,last);

    if(!length)
        return init;

    unsigned long const block_size=25;
    unsigned long const num_blocks=(length+block_size-1)/block_size;    <-1

    std::vector<std::future<T> > futures(num_blocks-1);
    thread_pool pool;

    Iterator block_start=first;
    for(unsigned long i=0;i<(num_blocks-1);++i)
    {
        Iterator block_end=block_start;
        std::advance(block_end,block_size);
        futures[i]=pool.submit(accumulate_block<Iterator,T>());     <-2
        block_start=block_end;
    }
    T last_result=accumulate_block<Iterator,T>()(block_start,last);
    T result=init;
    for(unsigned long i=0;i<(num_blocks-1);++i)
    {
        result+=futures[i].get();
    }
    result += last_result;
    return result;
}
```

When you compare this against listing 8.4, there are a couple of things to notice. First, you're working in terms of the number of blocks to use (`num_blocks` ❶) rather than the number of threads. In order to make the most use of the scalability of your thread pool, you need to divide the work into the smallest blocks that it's worth working with concurrently. When there are only a few threads in the pool, each thread will process many blocks, but as the number of threads grows with the hardware, the number of blocks processed in parallel will also grow.

You need to be careful when choosing the "smallest blocks that it's worth working with concurrently." There's an inherent overhead to submitting a task to a thread pool, having the worker thread run it, and passing the return value through a `std::future<>`, and for small tasks it's not worth the payoff. *If you choose too small a task size, the code may run more slowly with a thread pool than with one thread*.